

Implementing a Game with Containers

How §3–§5 of *Containers for Typed Agent AI* became **stalin**

Robin Langer

1 September 2026

1 The claim

A turn-based game is a container, and not by analogy.

P = the situations that can arise Rs = the moves that are legal in s

That is Ghani’s own third example, verbatim: “a data structure and its valid pointers — the positions you are allowed to navigate to.” And a *player* — human or machine — is exactly a program of type $(s : P) \rightarrow Rs$, which is §3’s “the programmer’s job”. A game engine that validates moves at run time is a game engine whose container it has not written down.

So the thing a game can demonstrate that a bookkeeping assistant cannot is sharper than “the types fit together”:

An illegal move should be *unconstructible*, not rejected.

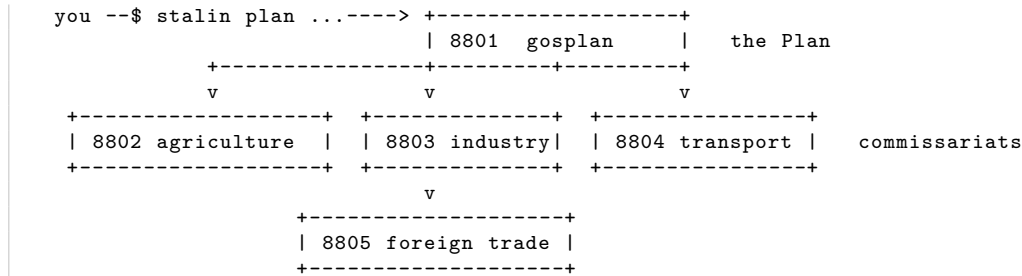
stalin is a resource-allocation game about the First Five-Year Plan, built to see how much of that is actually attainable in TypeScript. This note records what worked, what had to be discovered, and — §7 — what did not.

2 The tower is a command economy

§4 says an event handler is a morphism of containers: a pair

$$u : P \rightarrow Q, \quad f : (p : P) \rightarrow T(up) \rightarrow Rp,$$

control flowing down through u , data flowing back up through f . Relabel: Gosplan issues a quota to the Commissariat of Heavy Industry, which issues one to the mill, which reports what it made. That is not a metaphor for a command economy; it is one.



Each arrow is a spawned CLI process, not a function call, which matters for §7: between any two boxes everything the type system knew is destroyed, and each ingress has to re-establish it by validating rather than asserting.

3 The containers, written as their graphs

A container is presented not as a pair (Shape, Pos) but as its **graph**: the union of its fibres. That is what makes the type-level constructions compute, because a conditional type distributes over a union.

```
interface Fib<S, P> { readonly shape: S; readonly pos: P }
type Shape<C> = C extends Fib<infer S, unknown> ? S : never;
type PosAt<C, S> = C extends Fib<infer S1, infer P> ? (S extends S1 ? P : never) :
  never;
```

Each commissariat is then a declaration you can read against the server it describes — an indexed family, one line per index:

```
type AgricultureC =
  | Fib<{tag: "Harvest"; workforce: Workforce; tractors: number; year: number},
    HarvestReport>
  | Fib<{tag: "Winter"; ruralRations: Grain; industrialRations: Grain; year: number},
    WinterReport>
  | Fib<{tag: "Report"; year: number}, Dispatch>
  | Fib<{tag: "Census"}, CensusReturn>;
```

Two things are worth noticing. First, nothing says a server must correspond to exactly one container. Port 8803 serves three: the smelter, and separately

```
type TractorWorksC = Fib<{tag: "BuildTractors"; steel: Steel; plantCapacity: number;
  year: number}, TractorReport>;
type ArmamentsC = Fib<{tag: "BuildArmaments"; steel: Steel; year: number},
  ArmamentReport>;
```

They are split precisely so the Plan can take their *product* (§4.3). A sub-container is just a sub-family; the split costs nothing at run time and buys a combinator.

Second, **Report** and **Census** have different response types but the same standing — and that is where §6 comes back. See §6 below.

4 The four combinators

4.1 Tensor $\otimes$ — the three sectors

The fields, the mill and the railway all work through the same year, and all three owe a report. There is no partial year.

```
const fieldsAndMill = tensorC(agri, smelter);
const produce = tensorC(fieldsAndMill, transport);
```

Shapes multiply and *positions multiply*: a shape of **produce** carries a prompt for each sector, and its position carries a report from each. The **Promise.all** inside **tensorC** is not an optimisation, it is the definition.

4.2 Sequential composition $\triangleleft$ — the haul

What the railway is asked to move cannot be known until the fields have been reaped, because it depends on how much there is and on what grade it came in at. That is exactly $\triangleleft$: the composite's shape is a first prompt *together with a function* choosing the second from the first's reply.

```

const supply = seqC(produce, transport);

const supplied = await supply.run(
  supply.step(produceShape, (produced) => {
    const h = produced.left.left; // a HarvestReport, at this fibre
    const stateGrain = h.grain * take + s.grain;
    exp = resolveExport(h.grade, order.exportGrain); // <- the gate, SS5
    const offerPort = exp.choice === "none" ? 0 : /* ... */ ;
    return { tag: "Haul", railCapacity: s.railCapacity + produced.right.added,
             needIndustry: indMouths, offerPort, year };
  }),
  depth);

const harvest = supplied.first.left.left; // typed, uncast
const haul = supplied.second;

```

`produced.left.left` is a `HarvestReport` because that is the fibre we prompted, not because anybody asserted it. The application of `next` inside `seqC`'s runner is the whole content of $\triangleleft$: the composite shape carried a function, and that is where it is used.

4.3 Product $\times$ — tractors or armaments

The pattern §5 says the practitioners' catalogue had not named: prompt both sides, and owe a reply from exactly one. In the game it is the central strategic decision, because armaments are the only investment with no economic return whatever — they are worth something only if 1941 happens.

```

const build = productC(tractorWorks, armaments,
  // The policy reads the decision off the shape: whichever side was offered
  // the steel is the side that answers.
  ({ a }) => (a.steel > 0 ? "a" : "b"));

const made = await build.run(
  build.offer(
    { tag: "BuildTractors", steel: toTractors ? s.steel : 0, plantCapacity, year },
    { tag: "BuildArmaments", steel: toTractors ? 0 : s.steel, year }),
  depth);

if (made.side === "a") { s.tractors += made.value.built; }
else { s.warReserve += made.value.reserved; }

```

Narrowing on `made.side` is the elimination of $\text{Either}(Rp)(Tq)$. This is why the year's steel allocation was made a *decision* rather than a fraction: a split would have been the tensor, and calling it a product would have been false.

4.4 $\times \rightarrow \otimes$ — the railway to the port

The most satisfying place the algebra earned its keep. While the line to the port is narrow there is one trade delegation and one contract, so the sale is a *product* — grain or steel. Build enough rail and both can be shipped, so the sale becomes a *tensor*.

```

const narrowSale = productC(grainSale, steelSale, /* ... */);
const wideSale = tensorC(grainSale, steelSale);
// ...
const wide = throughputOf(s.railCapacity) === "wide";

```

The design goal was “transition from exporting grain to exporting steel, or export both”. In the algebra that last clause is not a bigger number; it is a different combinator. Same shapes, different positions: answer one, or answer both.

4.5 Sum + — present, but not as a call

`sumC` is never invoked in the game, and the note should say so plainly. Routing appears instead as the *shape type* of each container: a tagged union $\{\text{Harvest}\} + \{\text{Winter}\} + \dots$ is a coproduct, and its eliminator is the `switch` in each server’s dispatch. So the sum is doing real work, in the same place §3’s container always put it — but no composite container is built with it, and it would be an overstatement to claim otherwise.

5 The dependent gate

This is where the game does something the bookkeeping assistant could not.

The algebra gives Σ -types over *finite* index sets, because $\Sigma(s : S). B s = \bigcup_s \{s\} \times B s$ and a conditional type distributes over a union. So a rule expressible over a *grade* can be carried by the checker, and the same rule over a *tonnage* cannot. The game is therefore built to think in grades:

```
type ExportChoice<G extends HarvestGrade> =
  G extends "failure" ? "none"
: G extends "poor" ? "none" | "surplus"
: G extends "adequate" ? "none" | "surplus" | "full"
: "none" | "surplus" | "full" | "maximum";

function exportPlan<G extends HarvestGrade>(grade: G, choice: ExportChoice<G>): ...
```

`G` is inferred from the first argument, so `choice` is typed at *that* grade alone. The consequence:

```
exportPlan("bumper", "maximum"); // fine
exportPlan("failure", "maximum"); // does not compile
```

Exporting grain during a failed harvest is not a move the engine rejects. It is a move that cannot be written down — which was the whole ambition in §1.

The same trick carries the game’s arc:

```
type SteelTrade<P extends SteelPosition> =
  P extends "deficit" ? "none" | "buy"
: P extends "balanced" ? "none"
: "none" | "sell";
```

You begin in deficit and buy steel with grain at a punitive rate; you end in surplus and sell it. The transition from importing steel to exporting it is a **change of fibre**, not an `if`, and `balanced` is the hinge at which neither trade exists.

Introduction and elimination. The player types a string, so at the CLI boundary the choice is a `string` and must be checked — there is no avoiding that, and it is the same ingress discipline every hop needs anyway. What the gate buys is everything *past* that boundary. The check is a `switch` on the grade, and each branch is inside one fibre:

```
switch (grade) {
  case "failure": { const c = exportPlan("failure", "none"); /* ... */ }
  case "poor": { const c = exportPlan("poor", ok ? "surplus" : "none"); /* ... */ }
  // ...
}
```

That `switch` is the elimination rule of the finite Σ . Inside a branch the compiler knows which fibre it is in, and no illegal combination can be constructed from there on.

It is also historically apt. Quotas were set against reported categories, never measured tonnes. Making the type system coarse in the same way the bureaucracy was coarse is a coincidence, but a useful one: it means the discretisation the technique demands is not a distortion of the subject.

6 Shape and truth

§6 makes a caveat about the language model at the leaf: it is an untyped oracle, and presenting it as $(s : S) \rightarrow P s$ is a *promise* made good by a wrapper, not a fact about the model. The game relocates that caveat and makes it a mechanic.

Every commissariat holds two things the Plan cannot see: how well it is actually doing, and how far it will inflate a shortfall.

```
record(year: number, quota: number, delivered: number): Dispatch {
  const gap = Math.max(0, quota - delivered);
  const reported = delivered + gap * this.bias * (0.7 + 0.6 * this.noise());
  // ...
}
```

`Report` and `Census` return values of the same standing: both are well-typed positions of their container, both validated at ingress, both indistinguishable to the checker. One is true. The player plans against the other, and the divergence is revealed only at the reckoning.

The type of a dispatch guarantees its shape and says exactly nothing about its truth.

That is not a limitation of this encoding; it is what a type system is. A command economy is a typed tower whose leaves are untrusted oracles, and the types cannot save you — which is a serviceable summary both of §6 and of the subject matter.

7 Where it leaks

The $\triangleleft$ implemented is a sub-container of the paper's. Its fibres are indexed by *pairs* (S_A, S_B) : a first shape together with a function landing in one particular second shape. Ghani's $\triangleleft$ is larger, because there `next` may return different shapes of d for different positions, making the composite position a genuine $\Sigma(p : c.\text{Pos } s). d.\text{Pos } (vp)$ — a Σ over an *infinite* index, which does not distribute. Restricting `next` to a single fibre is exactly what buys the position type back as a plain product. What it costs is a branching `next`, which has a union return type and lies in no single fibre.

next must be pure, and the game wanted it not to be. The export decision is made inside `next` (it depends on the harvest grade) but is needed afterwards. Since `next` is a function of the position and returns only a shape, the decision is captured in a closure variable and read back after the run. That works and is flagged in the source, but it is a concession: the honest version would carry the decision in the composite shape.

The morphism is still not first class. The container is a value; the event handler over it is not. f must know which branch it is in to know what it owes, so the year is written one phase at a time. Each phase is monomorphic and therefore cast-free, but a genuinely first-class $\text{Cont}((P, R), (Q, T))$ would need the fibre treatment applied to morphisms as well.

deno check passed while the game was broken. When the library was vendored into this repository, all files typechecked and the first playthrough died at once, reporting that the delegate to port 8804 had exited with status 1. The reason is that `wire.ts` locates the CLI tool it spawns with

```
| new URL("./delegate.ts", import.meta.url) // resolved at run time, never checked
```

and that file had not been copied. The one file that makes each hop an actual process was the one file the compiler could not tell me was missing. Types hold within a process; the moment a boundary is a filesystem path or a socket, only running it tells you the truth.

8 Evidence

```
| $ deno check *.ts lib/*.ts          # 0 errors, including type-tests.ts
| $ ./playthrough.sh ...              # deterministic on the seed: a game is a regression test
```

`type-tests.ts` asserts the typed rules against the compiler. Every `@ts-expect-error` in it claims that the next line *is* an error, and TypeScript reports an unused directive as an error in its own right, so the file fails both if a legal move is rejected and if an illegal one is accepted. A negative suite that cannot fail is worth nothing, so it was mutation-tested: widening the "failure" fibre by one option makes it stop compiling.

That the types are right says nothing about whether the game is any good, so the design's central claim was checked by playing it. Three strategies, same seed:

strategy	starved	fallen	total dead	war
tractors, then armaments	0	14	14	won
tractors only, never armed	0	25	25	won
strip the villages, all armaments	53	8	61	won
build nothing	0	60	60	lost

Three of those win, and the two orderings disagree. Ranked by *soldiers* lost, stripping the villages is the best plan on the board — 8 fallen, because the smaller army that survives is lavishly armed. Ranked by *everyone* lost it is the worst. The game holds both numbers and declines to say which is the real one.

An earlier balance had brutality *losing* the war, and it took a remark from Robin to see what that was: cruelty punished by defeat is the comfortable version and not what happened. The thesis was not in the design, it was in a `Math.min` capping how far materiel may stand in for men, and in the constant giving the worth of an unarmed soldier. Two numbers were carrying an argument the simulation was then read back as having discovered.

One correction made while writing this document. Two of the four combinators, `supply` and `build`, were originally *declared and never run*: the haul and the steel allocation were direct calls that bypassed them. A document describing them as load-bearing would have been false, so the code was changed rather than the prose. It is worth recording because it is the characteristic failure mode of this kind of work: an algebra can be written down, typecheck, read beautifully, and not actually be in the path of anything.